

Unit-3

Q1) Explain Alpha - Beta Tree Search And Cutoff Procedure In Deatil With Example.

Introduction

Alpha–Beta Pruning is an optimization technique used in the **Minimax algorithm** in Artificial Intelligence.

It reduces the number of nodes evaluated in a game tree while giving the **same optimal result** as Minimax.

It is mainly used in two-player games such as:

- Chess
- Tic-Tac-Toe
- Checkers

The algorithm removes (prunes) branches that cannot affect the final decision.

1. Minimax Concept

In a game tree:

- **MAX player** tries to maximize the score.
- **MIN player** tries to minimize the score.

The tree is searched level by level until terminal nodes are reached.

Problem:

- Minimax checks **all nodes**, which takes more time.

Solution:

- Use **Alpha–Beta Pruning**.

2. Alpha and Beta Values

Alpha (α)

- Best value found so far for the **MAX** player.

Beta (β)

- Best value found so far for the **MIN** player.

3. Working Procedure

Step 1: Start from Root Node

Assign:

- $\alpha = -\infty$
- $\beta = +\infty$

Step 2: Traverse the Tree

- MAX nodes update α
- MIN nodes update β

Step 3: Check Cutoff Condition

If:

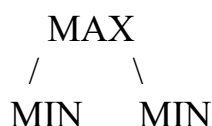
$$\alpha \geq \beta$$

then stop exploring that branch.

This is called **cutoff** or **pruning**.

4. Example of Alpha–Beta Pruning

Consider the following game tree:



$\begin{array}{cc} / \quad \backslash & / \quad \backslash \\ 3 \quad 5 & 6 \quad 9 \end{array}$

Step-by-Step Evaluation

Left MIN Node

Values:

- 3
- 5

MIN chooses smaller value:

$$\text{MIN}(3,5)=3$$

Now at root MAX:

- $\alpha = \max(-\infty, 3) = 3$

Right MIN Node

First child = 6

At MIN node: $\beta = 6$

Now compare: $\alpha = 3, \beta = 6$

Condition:

$$3 < 6$$

No pruning yet.

Next child = 9

$$\text{MIN chooses: } \text{MIN}(6,9)=6$$

Root MAX Node

$$\text{MAX chooses: } \text{MAX}(3,6) = 6$$

Final answer = 6

5. Cutoff Procedure

The cutoff procedure means stopping the search when further exploration cannot affect the final decision.

Conditions for Cutoff

At MAX node

- Cutoff occurs when: $\alpha \geq \beta$

At MIN node

- Cutoff also occurs when: $\alpha \geq \beta$

6. Advantages of Alpha–Beta Pruning

1. Reduces search space.
2. Faster than Minimax.
3. Evaluates fewer nodes.
4. Gives same optimal result as Minimax.
5. Useful in real-time games.

9. Disadvantages

1. Performance depends on node ordering.
2. Complex for very large trees.
3. Still expensive for deep searches.

Q2) What are the issues that need to be addressed for solving esp efficiently?

Explain the solutions to them

Issues to be Addressed for Solving Expert System Problems Efficiently (ESP)

Introduction

An **Expert System (ES)** is an Artificial Intelligence program that solves problems using the knowledge and reasoning ability of human experts.

To solve Expert System Problems (ESP) efficiently, several important issues must be addressed.

1. Knowledge Acquisition Problem

Issue

Collecting knowledge from human experts is difficult because:

- Experts may not explain their knowledge clearly.
- Knowledge may be incomplete or inconsistent.
- It takes a lot of time and effort.

Solution

- Use **knowledge engineers** to gather expert knowledge.
- Use interviews, questionnaires, and observation methods.

2. Knowledge Representation

Issue

Knowledge must be stored in a proper format so that the system can understand and use it efficiently.

Poor representation causes:

- Slow processing
- Incorrect reasoning
- Difficulty in updating knowledge

Solution

Use suitable representation techniques such as:

- Rules (IF–THEN rules)
- Semantic networks
- Frames
- Logic representation

Example:

IF fever AND cough

THEN disease = flu

3. Reasoning and Inference Efficiency

Issue

The system may take too much time to search through all rules and facts.

Solution

Use efficient inference methods:

- Forward chaining
- Backward chaining
- Alpha–Beta pruning (for search problems)
- Heuristic search techniques

This reduces unnecessary computations.

4. Handling Uncertainty

Issue

Real-world information is often uncertain or incomplete.

Example:

- Symptoms may not exactly match a disease.

Solution

Use uncertainty handling methods:

- Fuzzy logic
- Probability methods

These methods help the system make better decisions with incomplete data.

5. Knowledge Base Maintenance

Issue

Knowledge changes over time, so the knowledge base may become outdated.

Solution

- Regularly update rules and facts.
- Add new expert knowledge.

This keeps the system accurate and reliable.

6. User Interface Problem

Issue

A complicated interface makes the system difficult to use.

Solution

Design:

- Simple menus
- Easy input methods

A user-friendly interface improves interaction.

7. Explanation Facility

Issue

Users may not trust the system if it cannot explain its decisions.

Solution

Provide explanation features such as:

- Why a decision was made
- Which rules were used

Example:

Diagnosis = Flu

Because:

- Fever detected
- Cough detected

8. Speed and Performance

Issue

Large expert systems may become slow because of huge rule sets.

Solution

- Use optimized algorithms
- Use indexing and efficient search methods
- Reduce unnecessary rule checking

This improves system performance.

Q3) Explain in detail the concepts of back tracking and constraint propagation and solve the N-queen problem using these algorithms.

Backtracking and Constraint Propagation with N-Queen Problem

Introduction

The **N-Queen problem** is a famous problem in Artificial Intelligence and search algorithms. The objective is to place **N queens** on an **$N \times N$ chessboard** such that:

- No two queens attack each other.
- A queen can attack:
 - Horizontally
 - Vertically
 - Diagonally

The problem is solved efficiently using:

1. **Backtracking**
2. **Constraint Propagation**

1. Backtracking

Definition

Backtracking is a **search algorithm** that builds a solution step by step and removes choices that do not satisfy constraints.

It is also called:

- Depth First Search (DFS) based technique.

Working of Backtracking

1. Start placing queens row by row.
2. Check whether the current position is safe.
3. If safe:
 - Place the queen.
4. If not safe:
 - Try the next column.
5. If no valid position exists:
 - Remove previous queen (**backtrack**).
6. Continue until all queens are placed.

Algorithm of Backtracking

1. Start from first row.
2. Place queen in a safe column.
3. Move to next row.
4. Repeat until all queens are placed.
5. If no safe position exists:
 - remove previous queen
 - try another position.

2. Constraint Propagation

Definition

Constraint Propagation is a technique that reduces the search space by applying constraints before making decisions.

It eliminates impossible choices early.

Constraints in N-Queen Problem

A queen cannot be placed: In the same row ,In the same column ,In the same diagonal
Constraint propagation continuously updates allowed positions.

Working of Constraint Propagation

When a queen is placed:

- Mark attacked rows, columns, and diagonals.
- Remove invalid positions from consideration.

Solving 4-Queen Problem

We solve the **4-Queen problem** using Backtracking and Constraint Propagation.

Step-by-Step Solution

Step 1: Place Queen in Row 1

Place at:

(1,2)(1,2)(1,2)

Board:

_ Q _ _

_ _ _ _

Constraint propagation removes: Column 2 ,Diagonals attacked

Step 2: Row 2

Possible safe position:

(2,4)(2,4)(2,4)

Board:

_ Q _ _

_ _ _ Q

Remove attacked positions.

Step 3: Row 3

Safe position:

(3,1)(3,1)(3,1)

Board:

_ Q _ _

_ _ _ Q

Q _ _ _

_ _ _ _

Step 4: Row 4

Safe position:

(4,3)(4,3)(4,3)

Board:

_ Q _ _

_ _ _ Q

Q _ _ _

_ _ Q _

Final Solution

Queens are placed at:

(1,2),(2,4),(3,1),(4,3)(1,2), (2,4), (3,1), (4,3)(1,2),(2,4),(3,1),(4,3)

No queens attack each other.

Hence, the solution is correct.

Q4) Apply constraint satisfaction method to solve following Problem SEND + MORE = MONEY. (TWO + TWO = FOUR, CROSS+ ROADS= DANGER)

**Constraint Satisfaction Method for
SEND + MORE = MONEY**

Introduction

The problem:

SEND
+ MORE

MONEY

is a **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence.

In CSP:

- Each letter represents a unique digit.
- No two letters can have the same digit.
- Leading letters cannot be zero.

Step 1: Variables

Variables are:

S,E,N,D,M,O,R,Y,S,E,N,D,M,O,R,Y,S,E,N,D,M,O,R,Y

Step 4: Solve Step-by-Step

Using constraint propagation and backtracking:

Assign:

M=1M = 1M=1

From equation:

$S + M + C_3 = O + 10S + M + C_3 = O + 10$

Step 5: Verification

Substitute values:

9567
+ 1085

10652

Check:

$9567 + 1085 = 10652$

Final Solution

Letter Value

S	9
E	5
N	6
D	7
M	1
O	0
R	8
Y	2

Q5) Explain Min Max and Alpha Beta pruning algorithm for adversarial search with example.

Min-Max Algorithm and Alpha-Beta Pruning for Adversarial Search

Introduction

Adversarial search is used in Artificial Intelligence for two-player games such as:

- Chess
- Tic-Tac-Toe
- Checkers

One player tries to maximize the score while the other tries to minimize it.

The main algorithms used are:

1. Min-Max Algorithm
2. Alpha-Beta Pruning

1. Min-Max Algorithm

Definition

Min-Max is a decision-making algorithm used in game playing.

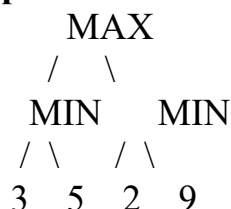
It finds the best move by assuming that:

- MAX player tries to maximize the score.
- MIN player tries to minimize the score.

Working of Min-Max

1. Generate the game tree.
2. Evaluate terminal nodes.
3. Apply:
 - MAX chooses maximum value.
 - MIN chooses minimum value.
4. Continue until root node is evaluated.

Example of Min-Max



Step 1: MIN Nodes

Left MIN:

$$\text{MIN}(3,5)=3$$

Right MIN:

$$\text{MIN}(2,9)=2$$

Step 2: MAX Node

$$\text{MAX}(3,2)=3$$

Final answer:

333

MAX player selects the left branch.

Advantages of Min-Max

1. Gives optimal solution.
2. Useful in game playing AI.
3. Simple and systematic.

Disadvantages

1. Slow for large game trees.

2. Examines many unnecessary nodes.
3. High time complexity.

2. Alpha-Beta Pruning

Definition

Alpha-Beta pruning is an optimization technique for Min-Max algorithm. It removes branches that cannot affect the final result.

Thus, fewer nodes are evaluated.

Alpha and Beta Values

Alpha (α)

Best value for MAX player.

$$\alpha = -\infty$$

Beta (β)

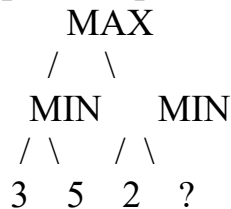
Best value for MIN player.

$$\beta = +\infty$$

Pruning occurs when:

$$\alpha \geq \beta$$

Example of Alpha-Beta Pruning



Step 1: Left MIN

$$\text{MIN}(3, 5) = 3$$

At root:

$$\alpha = 3$$

Step 2: Right MIN

First value:

2

So:

$$\beta = 2$$

Now:

$$\alpha = 3, \beta = 2$$

Since:

$$\alpha \geq \beta \quad 3 \geq 2$$

Remaining branch is pruned.

No need to evaluate “?”.

Advantages of Alpha-Beta Pruning

1. Faster than Min-Max.
2. Reduces search space.
3. Evaluates fewer nodes.

Q6) Explain with example graph coloring problem.

Graph Coloring Problem

Introduction

Graph Coloring is a Constraint Satisfaction Problem (CSP) in Artificial Intelligence and Graph Theory.

The objective is to color the vertices of a graph such that:

- No two adjacent vertices have the same color.
- Minimum number of colors should be used.

It is widely used in:

- Map coloring
- Timetable scheduling
- Register allocation
- Frequency assignment

Definition of Graph Coloring

Given a graph:

- Vertices (nodes)
- Edges (connections)

Assign colors to vertices so that connected vertices have different colors.

Example of Graph Coloring

Consider the graph:

```

  A
 / \
B---C
 \ /
  D

```

Edges:

- A–B
- A–C
- B–C
- B–D
- C–D

Coloring the Graph

Use colors:

- Red
- Blue
- Green

Step 1: Color A

Assign:

A=Red A = Red A=Red

Step 2: Color B

B is adjacent to A.

So:

B=Blue B = Blue B=Blue

Step 3: Color C

C is adjacent to:

- A (Red)
- B (Blue)

Therefore:

C=Green C = Green C=Green

Step 4: Color D

D is adjacent to:

- B (Blue)
- C (Green)

So D can take:

D=Red D = Red D=Red

Final Coloring

Vertex Color

A Red
B Blue
C Green
D Red

Important Condition

Adjacent vertices must not have same color.

Correct:

A(Red) ---- B(Blue)

Incorrect:

A(Red) ---- B(Red)

Types of Graph Coloring

1. Vertex Coloring
2. Edge Coloring
3. Face Coloring

Applications

1. Map coloring
2. Exam scheduling
3. Traffic signal control
4. Mobile network frequency allocation

Q7) How AI technique is used to solve tic-tac-toe problem.

AI Technique Used to Solve Tic-Tac-Toe Problem

Introduction

Tic-Tac-Toe is a two-player game played on a 3×3 board.

Artificial Intelligence solves this game using the **Minimax algorithm** and **adversarial search** techniques.

The players are:

- AI player (MAX)
- Human player (MIN)

The goal is to place three same symbols in a row, column, or diagonal.

Representation of Tic-Tac-Toe

Example board:

X | O | X

O | X |

| | O

AI searches all possible moves and chooses the best move.

AI Technique Used

1. Game Tree

AI creates a game tree containing:

- Current board state

- Possible future moves
- Winning and losing states

Each node represents a board configuration.

2. Minimax Algorithm

The Minimax algorithm is used for decision making.

MAX Player

- AI tries to maximize score.

MIN Player

- Opponent tries to minimize score.

Utility Values

Situation	Score
-----------	-------

AI wins	+1
---------	----

Opponent wins	-1
---------------	----

Draw	0
------	---

AI chooses the move with highest value.

Working Procedure

1. Generate all possible moves.
2. Apply Minimax recursively.
3. Evaluate terminal states.
4. Choose best possible move.

Example

Suppose board is:

X		O		X
---	--	---	--	---

O		X		
---	--	---	--	--

				O
--	--	--	--	---

AI checks possible positions.

If AI places X here:

X		O		X
---	--	---	--	---

O		X		
---	--	---	--	--

X				O
---	--	--	--	---

AI forms diagonal:

X-X-XX-X-XX-X-X

So AI wins.

Minimax selects this move because it gives highest score.

Alpha-Beta Pruning

To improve efficiency, AI may use Alpha-Beta pruning.

It:

- Removes unnecessary branches
- Reduces computation time
- Gives same optimal result

Advantages of AI in Tic-Tac-Toe

1. Always plays optimal move.

2. Can never lose if implemented correctly.
3. Fast decision making.
4. Demonstrates adversarial search concepts.

Q8) List All problem solving strategies. What is backtracking, explain with n queen problem, with Branch and bound or Backtracking.

1. Problem Solving Strategies in Artificial Intelligence

Problem-solving strategies are systematic approaches used by AI agents to find solutions.

Major strategies are:

1. **Generate and Test**
 - Generate possible solutions and test if they satisfy the goal.
2. **Breadth-First Search (BFS)**
 - Explores all nodes at the current depth before moving deeper.
3. **Depth-First Search (DFS)**
 - Goes deep into one path before backtracking.
4. **Hill Climbing**
 - Greedy local search that always moves towards better neighboring state.
5. **Best-First Search**
 - Uses heuristic to choose the most promising node.
6. *A Search**
 - Best-First Search using $f(n) = g(n) + h(n)$ (optimal).
7. **Means-Ends Analysis**
 - Reduces difference between current state and goal state (used in GPS).
8. **Problem Reduction**
 - Breaks complex problem into smaller sub-problems (AND-OR Graph).
9. **Constraint Satisfaction Problems (CSP)**
 - Variables with constraints (e.g., Graph Coloring, N-Queen).
10. **Backtracking**
 - Depth-first search with ability to undo (backtrack) wrong choices.
11. **Branch and Bound**
 - Used for optimization problems. Explores branches and prunes using bounds.

What is Backtracking?

Backtracking is a **depth-first search** technique used for solving constraint satisfaction problems. It builds the solution incrementally, and if at any point the partial solution violates constraints, it **backtracks** (undoes the last choice) and tries the next possible option.

Characteristics:

- Systematic trial and error.
- Saves time by abandoning invalid partial solutions early.
- Used when we need all possible solutions or one valid solution.

N-Queen Problem using Backtracking

Problem Statement: Place **N queens** on an $N \times N$ chessboard so that no two queens attack each other (i.e., no two queens in same row, column, or diagonal).

Backtracking Algorithm for N-Queen:

1. Start placing queens row by row (one queen per row).
2. For each row, try placing the queen in every column.
3. Check if the placement is **safe** (no attack from previously placed queens).
4. If safe → Place the queen and move to the next row.
5. If not safe → Try next column.

6. If no column is safe → **Backtrack** to the previous row and try next possibility.
7. If all queens are placed successfully → Solution found.

Example: 4-Queen Problem

We solve for N=4.

Step-by-step solution (one possible solution):

Initial Board: Empty 4x4

- Place Queen in Row 1, Col 2 → Safe
- Place Queen in Row 2, Col 4 → Safe
- Place Queen in Row 3, Col 1 → Safe
- Place Queen in Row 4, Col 3 → Safe → **Solution Found**

One Valid Solution:

text

```
. Q . .
. . . Q
Q . . .
. . Q .
```

Backtracking in Action:

- Suppose in Row 3 we tried Col 2 → Conflict (diagonal attack).
- Algorithm will **backtrack** to Row 2, remove queen from Col 4, and try next column.

Feature	Backtracking	Branch and Bound
Type	Search Technique	Optimization Technique
Used for	Finding feasible solution	Finding optimal solution
Pruning	Only when constraint violated	Uses upper/lower bounds to prune
Example	N-Queen, Sudoku	Travelling Salesman, Knapsack
Goal	Any valid solution	Best (minimum/maximum cost) solution

Q9) Explain Monte Carlo Tree Search with all steps and Demonstrate with one Example.

Q) Explain Monte Carlo Tree Search (MCTS) with all steps and Demonstrate with one Example.

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm used for decision-making in games and problems with large search spaces. It is widely used in **game playing AI** (especially in AlphaGo).

It combines the **random sampling** (Monte Carlo method) with **tree search**. Instead of exploring the entire game tree (which is impossible in games like Go), MCTS **intelligently builds** a search tree by focusing on promising moves.

Four Main Steps of MCTS (Repeated many times)

Each iteration of MCTS consists of **four phases**:

1. **Selection**
2. **Expansion**
3. **Simulation (Playout)**
4. **Backpropagation**

1. Selection

- Start from the **root node** (current game state).
- Traverse down the tree by selecting the **best child node** at each level.

- Use **Upper Confidence Bound (UCB1)** formula to balance **exploration** and **exploitation**:

$$UCB1 = \bar{x}_i + C \sqrt{\frac{\ln N}{n_i}} \quad UCB1 = \bar{x}_i + C \ln N$$

- $\bar{x}_i$ = Average win rate of node i
- N = Total visits of parent node
- n_i = Visits of current node i
- C = Exploration constant (usually ≈ 1.41)

- Continue until you reach a **leaf node** (a node that is not fully expanded).

2. Expansion

- From the selected leaf node, add **one or more child nodes** corresponding to possible legal moves.
- Usually, only **one child** is added per iteration (for efficiency).

3. Simulation (Playout / Rollout)

- From the newly expanded node, play the game **randomly** (or with a simple heuristic) until the game ends (terminal state).
- Record the result: **Win (+1)**, **Loss (-1)**, or **Draw (0)**.

4. Backpropagation

- Update the statistics (visit count and win count) of **all nodes** from the expanded node back up to the root.
- This updates the average reward of each node.

These 4 steps are repeated many times (hundreds or thousands of iterations) depending on time available. After all iterations, the best move is chosen from the root (usually the one with **highest visit count** or **highest win rate**).

Demonstration with Example: Tic-Tac-Toe

Let's assume it is **X's turn** and the board is almost empty.

Root Node: Empty board (X to move)

Iteration 1:

1. **Selection:** Only root node → Go to root.
2. **Expansion:** Add one possible move, e.g., X places in **Center**.
3. **Simulation:** Play randomly till end → Suppose **X wins**.
4. **Backpropagation:** Update Center node: Visits=1, Wins=1

Iteration 2:

1. **Selection:** Root → Select Center (high win rate).
2. **Expansion:** From Center, add another move (say X in Corner).
3. **Simulation:** Random play → Suppose **O wins**.
4. **Backpropagation:** Update both nodes.

Advantages of MCTS

- Handles very large search spaces efficiently.
- No need for domain-specific evaluation function.
- Balances exploration and exploitation.
- Anytime algorithm (can stop anytime and still give good result).

Limitations

- Can be slow if too few simulations.
- Poor performance in games with high randomness without modifications.
- Memory intensive for very deep trees.

Q10) Explain limitations of game search algorithm, Differentiate between stochastic and partial games AND

1. Limitations of Game Search Algorithms (Minimax, Alpha-Beta, etc.)

Game search algorithms like Minimax and Alpha-Beta have several practical limitations:

1. **Exponential Time Complexity:** The number of nodes grows exponentially with depth. Even with Alpha-Beta pruning, it becomes infeasible for deep games like Chess or Go.
2. **Limited Look-Ahead Depth:** Due to time constraints, only a few moves ahead can be searched (e.g., 4–6 plies in Chess), leading to poor decisions in complex positions.
3. **Requires Perfect Information:** These algorithms assume the full game state is known. They struggle in games with hidden information.

Feature	Stochastic Games	Partial Information Games
Information	Perfect Information	Imperfect / Hidden Information
Randomness	Involves chance (probability)	No randomness, but hidden moves/information
Example	Backgammon, Monopoly, Ludo	Poker, Bridge, Scrabble, Battleship
Chance Nodes	Yes (Dice, Cards, etc.)	No
Search Technique	Expectimax, MCTS	Strategy Fusion, Information Set Search

Q11) Explain How use of alpha and beta cut-offs will improve performance of mini max algorithm?

How Alpha-Beta Cut-offs Work?

Alpha-Beta pruning maintains two values during the search:

- **Alpha (α):** The best (highest) value that the **Maximizer** (MAX player) can guarantee.
- **Beta (β):** The best (lowest) value that the **Minimizer** (MIN player) can guarantee.

Rule of Pruning:

- If $\alpha \geq \beta$ at any node, then the remaining branches under that node can be **pruned** (cut off), because they will not be chosen.

How it Improves Performance of Mini-Max

Aspect	Mini-Max Algorithm	Mini-Max with Alpha-Beta Pruning	Improvement
Nodes Explored	All nodes	Much fewer nodes	Very High
Time Complexity	$O(b^d)$	Best case $\approx O(b^{\{d/2\}})$	Exponential saving
Space Complexity	Same	Same	No change
Final Result	Optimal	Same optimal result	No loss
Practical Speed	Slow for deep games	2 to 6 times faster	Major

Q12) Define is Constraint satisfaction problem, State the types of inconsistencies Solve the following Crypt Arithmetic Problem +MORE MONEY

1. Minimax Search Algorithm

Minimax is a recursive depth-first search algorithm used in **two-player, zero-sum, perfect information games** (e.g., Chess, Tic-Tac-Toe, Checkers).

- One player is **MAX** (maximizer) — tries to maximize the score.
- Other player is **MIN** (minimizer) — tries to minimize the score.

Minimax Algorithm (Pseudocode)

Python

```
function MINIMAX(node, depth, isMaximizingPlayer):
```

```
    // Base Case: Terminal node or depth limit reached
```

```
    if depth == 0 or node is a terminal state:
```

```
        return EVALUATION(node)    // Static evaluation function
```

```
    if isMaximizingPlayer:           // MAX Player's Turn
```

```
        maxEval = -∞
```

```
        for each possible move in node:
```

```
            child = apply_move(node, move)
```

```
            eval = MINIMAX(child, depth-1, False)
```

```
            maxEval = max(maxEval, eval)
```

```
        return maxEval
```

```
    else:                             // MIN Player's Turn
```

```
        minEval = +∞
```

```
        for each possible move in node:
```

```
            child = apply_move(node, move)
```

```
            eval = MINIMAX(child, depth-1, True)
```

```
            minEval = min(minEval, eval)
```

```
        return minEval
```

- At the **root node**, we call: MINIMAX(root, max_depth, True)
- MAX player selects the move with the **highest** minimax value.
- MIN player selects the move with the **lowest** minimax value.

2. How Alpha-Beta Cut-offs Improve the Performance of Minimax

Alpha-Beta Pruning (also called Alpha-Beta Cut-offs) is an **optimization technique** applied on Minimax algorithm. It **prunes** (cuts off) branches of the game tree that **will not affect** the final decision.

Key Parameters:

- **Alpha (α):** The best (highest) value that **MAX** can guarantee so far.
- **Beta (β):** The best (lowest) value that **MIN** can guarantee so far.

Pruning Condition:

- If at any point $\alpha \geq \beta$, then the remaining branches can be pruned.

Main Advantages of Alpha-Beta Cut-offs:

1. **Eliminates unnecessary branches** without affecting the final result.
2. **Reduces computation time** drastically.
3. **Allows deeper search** in the same time limit → stronger AI.

Q13) Define Game theory, Differentiate between stochastic and partial games with examples

Definition

Game Theory is a branch of Artificial Intelligence and mathematics that studies decision-making in competitive situations where two or more players interact according to specific rules.

In game theory:

- Players try to maximize their gains.
- Each player's decision affects the other players.

It is widely used in:

- Chess
- Tic-Tac-Toe
- Economics
- Business strategies

Difference Between Stochastic and Partial Information Games

Feature	Stochastic Games	Partial Information Games
Definition	Games that include randomness or chance events	Games where players do not have complete information
Uncertainty Source	Random events like dice/cards	Hidden information from opponents
Information Availability	Game state may be known	Complete game state is not visible
Use of Probability	High	Moderate
Decision Basis	Chance + strategy	Incomplete knowledge + strategy
Example	Ludo, Backgammon	Poker, Battleship

1. Stochastic Games

Definition

Stochastic games are games that involve random events or probability.

The outcome depends on:

- Player actions
- Chance events

Example: Ludo

- Dice roll determines movement.
- Players cannot fully control outcomes.

Another example:

- Backgammon

2. Partial Information Games

Definition

Partial information games are games in which players cannot see the complete game state. Some information remains hidden.

Example: Poker

- Players cannot see opponents' cards.
- Decisions are made with incomplete information.

Another example:

- Battleship

Q14) Define is Constraint satisfaction problem, State the types of consistencies Solve the followiig Crypt Arithmetic Problem. B A S E + B A L L G A M E S

Definition

A Constraint Satisfaction Problem (CSP) is a problem in Artificial Intelligence where:

- Variables are assigned values
- Subject to certain constraints or conditions

A CSP consists of:

1. Variables
2. Domains (possible values)
3. Constraints

Examples:

- Sudoku
- Map coloring
- Crypt arithmetic problems

Types of Consistencies

1. Node Consistency

A variable satisfies its unary constraints.

2. Arc Consistency

For every value of one variable, there exists a valid value in another variable.

3. Path Consistency

Consistency is maintained among three variables together.

4. k-Consistency

Consistency is checked among k variables.

Crypt Arithmetic Problem

BASE

+ BALL

GAMES

Each letter represents a unique digit.

Constraints

- All letters have different digits.
- Leading letters cannot be zero.
- Digits are assigned so that arithmetic addition is correct.

Variables:

B,A,S,E,L,G,M,B, A, S, E, L, G, M,B,A,S,E,L,G,M

Solution Using CSP

By applying constraint propagation and backtracking:

Letter Digit

B	7
A	4
S	8
E	3
L	5
G	1

Letter Digit

M 9

Verification

7483

+ 7455

14938

$7483 + 7455 = 14938$

Hence:

BASE + BALL = GAMES

is satisfied.

Q15) Define Game Theory. What are the components, defining game as a Search Problem? Draw a Game Tree for tic-tac-toe.

Definition

Game Theory is a branch of Artificial Intelligence and mathematics that studies strategic decision-making between two or more players where each player tries to maximize gains and minimize losses.

It is mainly used in:

- Chess
- Tic-Tac-Toe
- Poker

Components Defining Game as a Search Problem

A game can be represented as a search problem using the following components:

1. Initial State

The starting configuration of the game.

Example:

- Empty Tic-Tac-Toe board.

2. Players

The participants in the game.

Example:

- MAX player
- MIN player

3. Actions / Moves

Possible legal moves available to a player.

Example:

- Placing X or O in an empty cell.

4. Result Function

Defines the new state after applying a move.

5. Terminal Test

Checks whether the game is over.

Conditions:

- Win
- Loss
- Draw

6. Utility Function

Assigns numerical values to outcomes.

Example:

Result Value

Win +1

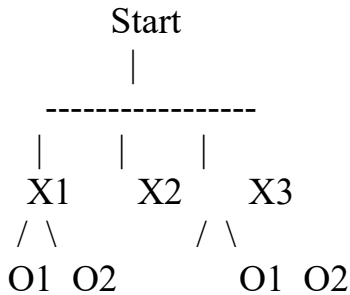
Loss -1

Draw 0

Game Tree for Tic-Tac-Toe

A game tree represents all possible moves of the game.

Example:



- Root node = Initial board
- Branches = Possible moves
- Nodes = Game states

Q16) What do you understand by Constraint Propagation? Write short notes on Node Consistency and Arc Consistency.

Definition

Constraint Propagation is a technique used in Constraint Satisfaction Problems (CSP) to reduce the search space by eliminating values that do not satisfy constraints.

It repeatedly applies constraints to variables so that invalid values are removed early.

Advantages

1. Reduces unnecessary searching
2. Improves efficiency
3. Helps in faster problem solving

Examples:

- Sudoku
- N-Queen problem
- Map coloring

Node Consistency

Definition

A variable is node consistent if all values in its domain satisfy the unary constraints of that variable.

Unary constraint means a constraint involving only one variable.

Example

Suppose:

$X > 0$

Domain of XXX:

$\{-2, -1, 0, 1, 2\} \setminus \{-2, -1, 0, 1, 2\} \setminus \{-2, -1, 0, 1, 2\}$

After applying node consistency:

$\{1, 2\} \setminus \{1, 2\} \setminus \{1, 2\}$

Invalid values are removed.

Arc Consistency

Definition

A variable is arc consistent if for every value of one variable, there exists a valid value in the connected variable satisfying the constraint.

It works on constraints between two variables.

Example

Constraint:

$$X < Y \quad X < Y \quad X < Y$$

Domains:

$$X = \{1, 2, 3\} \quad X = \{1, 2, 3\} \quad X = \{1, 2, 3\} \quad Y = \{2\} \quad Y = \{2\} \quad Y = \{2\}$$

Since $X=2$, $X=2$, $X=2$ and $X=3$, $X=3$, $X=3$ do not satisfy $X < Y$, $X < Y$, $X < Y$,

Updated domain:

$$X = \{1\} \quad X = \{1\} \quad X = \{1\}$$

Thus, arc consistency removes inconsistent values.

Q17) Explain how Minimax and alpha-beta algorithms change for two-players, non zero-sum games in which each player has his or her own utility function.

Minimax and Alpha-Beta in Two-Player Non Zero-Sum Games

Introduction

In a **zero-sum game**, one player's gain is exactly equal to the other player's loss.

Example:

- Chess
- Tic-Tac-Toe

But in a **non zero-sum game**, both players can:

- Win partially
- Lose partially
- Have independent utility values

Each player has his or her own utility function.

Example:

- Business negotiation
- Resource sharing
- Cooperative games

Minimax in Non Zero-Sum Games

Traditional Minimax

In zero-sum games:

- MAX tries to maximize utility.
- MIN tries to minimize MAX's utility.

Single utility function is used.

Change in Non Zero-Sum Games

In non zero-sum games:

- Each player has a separate utility function.
- Players do not always have completely opposite goals.

Instead of one value:

$$\text{Utility} = (U_1, U_2) \quad \text{Utility} = (U_1, U_2) \quad \text{Utility} = (U_1, U_2)$$

where:

- U_1 = utility of Player 1
- U_2 = utility of Player 2

Example

Suppose final outcomes are:

Move	Player A	Player B
X	8	5
Y	6	7

- Player A prefers X
- Player B prefers Y

The algorithm evaluates both utilities before selecting moves.

Alpha-Beta Pruning in Non Zero-Sum Games

Traditional Alpha-Beta

In zero-sum games:

- Alpha = best value for MAX
- Beta = best value for MIN
- Pruning occurs when:

$$\alpha \geq \beta$$

Changes in Non Zero-Sum Games

In non zero-sum games:

- Utilities are independent.
- One player's gain is not equal to another's loss.

Therefore:

- Simple alpha-beta pruning cannot be directly applied.
- Multiple utility values must be considered.

Limitations

1. More complex than zero-sum games.
2. Harder to prune branches.
3. Requires more computation.
4. Cooperation between players may affect decisions.

Q18) Define Constraint Satisfaction Problem. Explain Map Coloring Example Problem. . Formulate the Map Coloring Problem as CSP.

Definition

A Constraint Satisfaction Problem (CSP) is a problem in Artificial Intelligence where values are assigned to variables while satisfying given constraints.

A CSP consists of:

1. Variables
2. Domains
3. Constraints

Examples:

- Sudoku
- N-Queen problem
- Map coloring

Map Coloring Problem

Definition

In the map coloring problem, different colors are assigned to regions of a map such that no two adjacent regions have the same color.

This problem is solved using CSP techniques.

Example of Map Coloring

Consider four regions:

A
 /\n
 B---C
 \/\n
 D

Adjacent regions:

- A adjacent to B and C
- B adjacent to C and D
- C adjacent to D

Formulation of Map Coloring as CSP

1. Variables

$X = \{A, B, C, D\}$ $X = \{A, B, C, D\}$ $X = \{A, B, C, D\}$

Each variable represents a region.

2. Domain

Available colors:

$D = \{\text{Red}, \text{Green}, \text{Blue}\}$ $D = \{\text{Red}, \text{Green}, \text{Blue}\}$ $D = \{\text{Red}, \text{Green}, \text{Blue}\}$

3. Constraints

Adjacent regions cannot have same color.

Constraints are:

$A \neq B$ $A \neq C$ $B \neq C$ $B \neq D$ $C \neq D$
 $A \neq B$ $A \neq C$ $B \neq C$ $B \neq D$ $C \neq D$

Solution

Assign colors:

Region Color

A Red
 B Green
 C Blue
 D Red

This satisfies all constraints because adjacent regions have different colors.

Applications of Map Coloring

1. Timetable scheduling
2. Frequency assignment
3. Register allocation
4. Geographic map coloring